



AIKR PROJECT

Smart Email Management AI Agent

Complete A-to-Z Project Guide

From Scratch — For First-Time Project Students

Student	Aswarth
College	IIIT Sricity
Program	B.Tech AI & Data Science — 2nd Year
Project Code	AIKR — S2026
Language	Python 3.10+
Guide Version	Week-by-Week Master Plan

This guide contains everything you need: what to build, what to learn, how to code it, what to submit, and how to score full marks.

CHAPTER 1 | BIG PICTURE — What is this project about?

1.1 What You Are Building

You are building a **Smart Email Management AI Agent** — a Python program that connects to your Gmail, reads your emails, understands them using Artificial Intelligence, and then takes intelligent actions like classifying, summarizing, prioritizing, and drafting replies. Think of it as building your own personal AI assistant for your inbox.

In Simple Words

- Your program will **READ** your Gmail emails automatically.
- It will **UNDERSTAND** what each email is about using AI.
- It will **CLASSIFY** them: Urgent / Work / Personal / Spam / Finance.
- It will **SUMMARIZE** long emails into 2-3 short sentences.
- It will **DRAFT** replies automatically.
- It will **PRIORITIZE** — tell you which email needs attention first.

1.2 Why This Project Matters (For Marks)

Your project guidelines ask for a real AI Agent that receives inputs, reasons, and performs actions. Your email agent does exactly this:

Guideline Requirement	How Your Agent Satisfies It
Receive inputs	Reads real Gmail emails via Gmail API
Reason logically	Uses rule-based scoring + LLM reasoning
Perform actions	Classifies, summarizes, drafts replies
AI techniques	ML classification + LLM + rule-based reasoning
Search algorithms	BFS-style email graph traversal for threads
Agent type	Goal-based + utility-based combined agent

1.3 Your Final Project Will Have These Parts

- **Gmail Connection Module** — Connects to Gmail, fetches real emails
- **Email Preprocessor** — Cleans email text (removes HTML, noise)
- **AI Classifier** — Labels emails: Urgent / Work / Personal / Spam
- **Summarizer** — Compresses long emails into short summaries
- **Reply Generator** — Drafts professional replies automatically
- **Priority Engine** — Scores emails 1–10 by urgency
- **AI Agent Core** — Decides what action to take for each email

- **Streamlit UI** — Visual dashboard to see everything (BONUS +5)

Marks Breakdown (100 total)

- Problem Definition: 10 marks — clearly explain what and why
- Agent Design & Architecture: 10 marks — explain your agent type and design
- AI Logic / Algorithms: 30 marks — THIS IS THE BIGGEST SECTION
- Implementation Quality: 30 marks — working code + clean structure
- Testing & Demo: 10 marks — show 3 test cases
- Report Quality: 10 marks — professional PDF report
- BONUS: UI (Streamlit) +5, Innovative Features +5, From Scratch +20

CHAPTER 2 | LEARNING PLAN — What to learn, in what order

2.1 What You Already Know vs What You Need

What You Know	How It Helps	Effort Needed
BFS / DFS / A* search	AI Agent decision logic	None — already known
Machine Learning basics	Email classification	Small review
Deep Learning (CNN, RNN)	Understand LLM foundations	None
Hashing, Trees, Sorting	Email indexing & lookup	None
Python basics	All code is in Python	Small review if rusty

What You Need to Learn	How Hard?	When to Learn?	Time Needed
Gmail API + OAuth2	Easy	Week 1	3 hours
Python file handling + JSON	Easy	Week 1	2 hours
NLP basics (tokenize, clean)	Easy-Medium	Week 2	4 hours
Hugging Face Transformers	Medium	Week 2	5 hours
OpenAI / Gemini API calls	Easy	Week 3	2 hours
Prompt Engineering	Easy	Week 3	3 hours
LangChain Agent basics	Medium	Week 4	6 hours
Streamlit UI	Easy	Week 6	3 hours

Key Insight

- You do NOT need to learn everything before starting.
- Learn each topic 1-2 days before building that week's module.
- This is called Learning by Doing — most efficient method.
- Total new learning needed: ~28 hours across 6 weeks = very manageable.

2.2 Best YouTube Videos (Watch in This Order)

Watch each video in the week it is listed. Do not watch all at once.

Week	Topic	Search This on YouTube	Duration
------	-------	------------------------	----------

1	What is REST API?	Fireship REST API explained	5 min
1	Gmail API full setup	Tech With Tim Gmail API Python	30 min
1	Google Cloud Console	Google Cloud Console tutorial 2024	15 min
1	Python JSON	Corey Schafer Python JSON tutorial	20 min
2	NLP basics	Sentdex NLP Python tutorial	40 min
2	Hugging Face intro	Hugging Face transformers tutorial beginner	35 min
3	OpenAI API Python	Tech With Tim OpenAI API tutorial	25 min
3	Prompt Engineering	Andrew Ng Prompt Engineering course	2 hrs
4	LangChain basics	Greg Kamradt LangChain tutorial	60 min
4	LangChain agents	LangChain agents tools Python 2024	45 min
6	Streamlit tutorial	Streamlit tutorial Python Misra	30 min

CHAPTER 3 | WEEK-BY-WEEK PLAN — Exactly what to do each day

WEEK 1

Gmail Setup + Email Fetching

Day	What To Do
Mon	Install Python, libraries. Run setup check file. Watch Fireship REST API video (5 min).
Tue	Set up Google Cloud Console. Enable Gmail API. Download credentials.json. Watch Tech With Tim Gmail video.
Wed	Run step2_gmail_auth.py. Connect to Gmail. Token saved. Test that labels show.
Thu	Run step3_fetch_emails.py. See your real emails in terminal. emails.json saved.
Fri	Run step4_clean_emails.py. Understand HTML cleaning and regex. emails_clean.json saved.
Sat	Run full pipeline (step5). Write 5 reflection answers in notebook.
Sun	Review all 5 files. Make sure you can EXPLAIN every line to a person.

WEEK 2

NLP + Email Classification

Day	What To Do
Mon	Watch Sentdex NLP basics video. Learn: tokenization, stopwords, TF-IDF.
Tue	Watch Hugging Face transformers tutorial. Install transformers library.
Wed	Build classifier.py — rule-based classifier first (keyword matching).
Thu	Add ML classifier using BERT (distilbert-base-uncased from Hugging Face).
Fri	Test classification on your 15 real fetched emails. Check accuracy.
Sat	Combine Week 1 + Week 2 into one pipeline. Save classified emails JSON.
Sun	Review. Can you explain: What is BERT? What is tokenization? Practice out loud.

WEEK 3

Summarization + Reply Generation

Day	What To Do
Mon	Get free Gemini API key (ai.google.dev). Watch OpenAI API tutorial.
Tue	Build summarizer.py. Write a prompt that summarizes email in 2-3 sentences.
Wed	Test summarizer on 5 different email types. Refine the prompt.
Thu	Build reply_generator.py. Write prompt for professional reply drafting.
Fri	Test reply generation on urgent email, work email, personal email.
Sat	Combine Weeks 1-2-3 into full pipeline. All modules connected.
Sun	Review. Write in notebook: What is prompt engineering? Why does it matter?

WEEK 4

AI Agent Core (LangChain)

Day	What To Do
Mon-Tue	Watch Greg Kamradt LangChain tutorial (60 min). Understand: Agent, Tools, Chain.
Wed	Build agent.py — create LangChain agent with 4 tools: classify, summarize, reply, priority.
Thu	Test agent on 5 emails. Agent auto-decides which tool to call.
Fri	Add BFS-based thread analyzer (this satisfies the Search Algorithm requirement for marks).
Sat	Full pipeline test: Gmail → Clean → Agent → Output. Fix bugs.
Sun	Review. Practice explaining agent architecture out loud (viva prep).

WEEK 5

Testing + Integration

Day	What To Do
Mon-Tue	Write 3 formal test scenarios (required for 10 marks). Document input, expected output, actual output.
Wed	Integration testing: run full pipeline with 15 real emails. Log all results.
Thu	Bug fixing day. Fix any errors found in integration testing.
Fri	Performance test: measure time taken per email. Optimize slow parts.
Sat-Sun	Write README.md and requirements.txt. Organize project folder structure.

WEEK 6

UI + Report + Submission

Day	What To Do
Mon-Tue	Build Streamlit UI. Watch Streamlit tutorial. Show email list + analysis results.
Wed-Thu	Write project report PDF (8-10 pages, follow the template in Chapter 7).
Fri	Record demo video (3-5 min). Show: problem, system working, agent decisions.
Sat	Final review. Check all submission items: ZIP, PDF report, video.
Sun	Submit before deadline. Celebrate.

CHAPTER 4 | TECH STACK & ARCHITECTURE — Everything you will

4.1 Complete Technology Stack

Layer	Technology	Purpose	Difficulty
Language	Python 3.10+	Everything is in Python	You know this
Email Access	Gmail API + OAuth2	Fetch real emails securely	Easy
Text Cleaning	BeautifulSoup, re (regex)	Remove HTML and noise	Easy
NLP / ML	Hugging Face Transformers	Email classification (BERT)	Medium
LLM (AI brain)	Google Gemini API (free)	Summarize + draft replies	Easy
AI Agent	LangChain	Agent that decides actions	Medium
Search Algorithm	BFS (custom Python)	Thread traversal (for marks)	You know this
Storage	JSON / SQLite	Save emails and results	Easy
UI (Bonus)	Streamlit	Web dashboard interface	Easy
Report	PDF (manual writing)	8-10 page project report	Writing

4.2 Agent Architecture (Explain This in Viva)

Your agent is a **Goal-Based + Utility-Based hybrid agent**. Here is what that means:

- Goal-based: The agent has a clear goal — process every email and take the right action.
- Utility-based: It assigns a priority score (utility value) to decide which email is most important.
- The agent uses Tools (LangChain tools) to perform each action.

Component	Agent Term	In Your System
Environment	Where agent lives	Gmail inbox
Percepts (inputs)	What agent senses	Email: subject, sender, body, date
Reasoning Engine	Agent brain	LangChain Agent + Gemini LLM
Actions (outputs)	What agent does	Classify, summarize, reply, prioritize
Goal	What agent wants	Process all emails correctly
Utility function	How to measure success	Priority score 1-10 per email

Memory	Past knowledge	JSON file of processed emails
--------	----------------	-------------------------------

4.3 Folder Structure (Submit Exactly This)

```
project/
├── src/
│   ├── gmail_auth.py ← Gmail OAuth2 connection
│   ├── email_fetcher.py ← fetch and extract emails
│   ├── email_cleaner.py ← clean email text
│   ├── classifier.py ← classify email category
│   ├── summarizer.py ← summarize email body
│   ├── reply_generator.py ← draft email replies
│   ├── priority_engine.py ← score email urgency
│   ├── bfs_thread.py ← BFS thread analyzer
│   ├── agent.py ← LangChain AI Agent
│   └── app.py ← Streamlit UI
├── data/
│   ├── emails.json ← raw fetched emails
│   ├── emails_clean.json ← cleaned emails
│   └── README.md ← project description
└── requirements.txt ← all library names
```

CHAPTER 5 | AI LOGIC & ALGORITHMS — The most important chapter

IMPORTANT

- AI Logic is worth 30 marks — the BIGGEST section.
- You must implement at least ONE search algorithm + ONE ML technique.
- Your project implements BOTH — which gives you maximum marks.

5.1 Search Algorithm: BFS for Email Thread Analysis

BFS (Breadth-First Search) is used to traverse email conversation threads. An email thread is like a tree: one parent email, multiple replies, sub-replies. BFS explores all replies at the same level before going deeper.

How to explain it in viva:

- Each email in a thread = a node in a graph
- Thread replies = edges connecting nodes
- BFS finds all related emails in the thread level by level
- This helps the agent understand full conversation context before deciding action

BFS Thread Analyzer — Core Logic

```
from collections import deque def bfs_thread_analyzer(email_id, thread_map): # thread_map = {email_id: [reply_id1, reply_id2, ...]} visited = set() # track visited nodes (email IDs) queue = deque([email_id]) # BFS queue, start from root thread = [] # result: all emails in this thread while queue: current = queue.popleft() # get next email to process if current in visited: continue visited.add(current) thread.append(current) replies = thread_map.get(current, []) # get all replies for reply in replies: if reply not in visited: queue.append(reply) # add reply to queue return thread # list of all email IDs in conversation
```

5.2 Machine Learning: BERT Email Classification

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained deep learning model for understanding text. You use it to classify emails into categories. You use a pre-trained version from Hugging Face — no training from scratch needed.

- Input: email subject + body text (cleaned)
- Processing: BERT tokenizes text → creates embeddings (number vectors)
- Output: category label (Urgent / Work / Personal / Finance / Spam)
- Why BERT: it understands context, not just keywords

5.3 Rule-Based Reasoning: Priority Scoring

A rule-based system uses IF-THEN logic to make decisions. Your priority engine uses rules like: if sender is professor AND subject contains 'deadline' → score = 9.

- Score 9-10: Keywords like 'urgent', 'deadline', 'immediately' from important senders
- Score 6-8: Work emails with action items
- Score 3-5: Personal emails, informational content
- Score 1-2: Promotional, spam, newsletters

5.4 LLM Reasoning: Gemini API

The Gemini API (large language model) provides the reasoning and generation capability. It does summarization and reply drafting using natural language prompts.

- Summarization prompt: 'Summarize this email in 2-3 sentences focusing on action items.'
- Reply prompt: 'Write a professional 3-5 sentence reply to this email as a student.'
- Why this qualifies as AI: LLMs use transformer architecture — deep learning at its core.

Summary: AI Techniques Used (List This in Report)

- 1. Search Algorithm: BFS for email thread traversal (satisfies mandatory requirement)
- 2. Machine Learning: BERT-based text classification (Hugging Face)
- 3. Rule-Based Reasoning: Priority scoring with weighted rules
- 4. LLM Reasoning: Gemini API for summarization and reply generation
- 5. Agent Decision Making: LangChain tool selection logic

CHAPTER 6 | COMPLETE CODE GUIDE — All files, line by line

6.1 Install All Libraries (Run Once)

```
pip install google-auth google-auth-oauthlib google-api-python-client pip install
beautifulsoup4 transformers torch pip install langchain langchain-google-genai
google-generativeai pip install streamlit pandas
```

6.2 Priority Engine (New — Not in Week 1)

```
# priority_engine.py URGENT_WORDS = ['urgent', 'deadline', 'immediately', 'asap', 'today', 'last chance']
WORK_WORDS = ['meeting', 'project', 'assignment', 'submit', 'report', 'task'] SPAM_WORDS =
['offer', 'discount', 'free', 'click here', 'win', 'limited time'] KNOWN_SENDERS =
['professor', 'faculty', 'admin', 'placement', 'iiit', 'hr'] def score_priority(email): score = 5 # default
medium priority subject = email['subject'].lower() body = email['body_clean'].lower() sender =
email['sender'].lower() text = subject + ' ' + body # Rule 1: urgent keywords → increase score for word in
URGENT_WORDS: if word in text: score += 2 # Rule 2: known important senders → increase score for s in
KNOWN_SENDERS: if s in sender: score += 1 # Rule 3: spam keywords → decrease score for word in SPAM_WORDS:
if word in text: score -= 2 # Rule 4: unread email → slight increase if email.get('is_unread'): score += 1
score = max(1, min(10, score)) # clamp between 1 and 10 return score
```

6.3 Gemini API (Summarize + Reply)

```
# summarizer.py import google.generativeai as genai genai.configure(api_key='YOUR_GEMINI_API_KEY') # get
free at ai.google.dev model = genai.GenerativeModel('gemini-pro') def summarize_email(email): prompt =
f""" Summarize this email in 2-3 sentences. Focus only on: what action is needed, who sent it, and when. Be
direct. No extra words. Subject: {email['subject']} From: {email['sender']} Body:
{email['body_clean'][:800]} """ response = model.generate_content(prompt) return response.text def
draft_reply(email): prompt = f""" Write a professional, polite reply to this email. Keep it 3-5 sentences.
Sound like a B.Tech student. Subject: {email['subject']} From: {email['sender']} Body:
{email['body_clean'][:600]} """ response = model.generate_content(prompt) return response.text
```

6.4 LangChain Agent Core

```
# agent.py from langchain.agents import initialize_agent, Tool from langchain_google_genai import
ChatGoogleGenerativeAI from classifier import classify_email from summarizer import summarize_email,
draft_reply from priority_engine import score_priority # Define each capability as a Tool tools = [
Tool(name='Classify', func=classify_email, description='Classify email as
Urgent/Work/Personal/Finance/Spam'), Tool(name='Summarize', func=summarize_email, description='Summarize
the email body in 2-3 sentences'), Tool(name='Reply', func=draft_reply, description='Draft a professional
reply to the email'), Tool(name='Priority', func=score_priority, description='Score email urgency from 1
to 10'), ] llm = ChatGoogleGenerativeAI(model='gemini-pro', google_api_key='YOUR_KEY') agent =
initialize_agent(tools, llm, agent='zero-shot-react-description') def process_email_with_agent(email):
task = f'Process this email: {email["subject"]} from {email["sender"]}' result = agent.run(task) return
result
```

CHAPTER 7 | PROJECT REPORT TEMPLATE — 8-10 pages — exactl

Important

- Your report must be a PDF, 8-10 pages, submitted with your ZIP file.
- Follow the structure below EXACTLY. Each section has a marks value.
- Write in simple, clear English. Do not copy-paste from the internet.

1. Problem Statement (10 marks)

- Write: 'Email management is a critical challenge for students and professionals...'
- Explain: People receive 100+ emails per day; manually reading all is inefficient.
- State the problem: No intelligent system exists to auto-classify and summarize emails.
- State your solution: An AI Agent that connects to Gmail, classifies, summarizes, prioritizes.
- Tip: Write 1 page. Be specific. Mention IIIT Sricity context (placement emails, faculty emails).

2. Agent Architecture (10 marks)

- Draw a diagram: Gmail → Fetcher → Cleaner → Classifier → Summarizer → Agent → Output
- State agent type: 'Goal-based + Utility-based hybrid agent'
- Define: Environment (Gmail inbox), Percepts (email text), Actions (classify/summarize/reply)
- Explain the utility function: priority score 1-10 assigned to each email
- Tip: Include the architecture diagram as an image. Draw it in draw.io (free tool).

3. AI Techniques Used (30 marks — most important)

- Section 3.1: BFS Search — explain thread traversal with pseudocode
- Section 3.2: Machine Learning — explain BERT classification with diagram
- Section 3.3: Rule-Based Reasoning — show priority scoring rules table
- Section 3.4: LLM Reasoning — explain Gemini API usage with example prompt
- Section 3.5: Agent Decision Making — explain LangChain tool selection
- Tip: For each technique, write: What it is → Why used → How implemented → Example output

4. System Design (part of Implementation — 30 marks)

- List all 9 source files and what each does (1 sentence each)
- Show the folder structure (copy from Chapter 4 of this guide)
- Explain the data flow: email enters → cleaned → classified → summarized → stored
- Show the JSON structure of one processed email

5. Implementation Details (30 marks continued)

- Show 3-4 key code snippets with explanations (copy from your actual code)
- Explain the Gmail OAuth2 setup in 1 paragraph
- Explain the BERT classification in 1 paragraph
- Explain the LangChain agent tools in 1 paragraph
- Tip: Do NOT paste entire files. Paste only the most important 15-20 lines.

6. Results & Testing (10 marks)

- Test Scenario 1: Urgent email from professor → expected: Urgent + score 9 + summary
- Test Scenario 2: Spam email (Swiggy offer) → expected: Spam + score 2 + no reply
- Test Scenario 3: Placement email → expected: Work + score 7 + draft reply
- For each scenario: show actual input email, show actual output, compare to expected
- Tip: Take screenshots of your terminal output for each test.

7. Limitations

- Gmail API rate limit: can fetch maximum 100 emails per minute
- BERT classification accuracy is ~85% — some emails may be misclassified
- Gemini API requires internet connection — offline mode not supported
- Does not handle emails in languages other than English

8. Conclusion

- Write: what you built, what you learned, what worked well
- Mention: AI techniques used, challenges faced, how they were solved
- Future improvements: multi-language support, calendar integration, voice assistant

9. References

- Gmail API Documentation — developers.google.com/gmail/api
- Hugging Face BERT — huggingface.co/docs/transformers
- LangChain Documentation — docs.langchain.com
- Google Gemini API — ai.google.dev
- Any textbook or research paper you referenced

10. AI Usage Declaration (Mandatory)

- Tool: Claude AI — Purpose: Architecture design, code review, explanation of concepts
- Tool: Gemini API — Purpose: Core AI functionality (summarization, replies) — part of project
- Tool: GitHub Copilot (if used) — Purpose: Code suggestions and debugging
- Note: All code was understood and can be explained by the student.

CHAPTER 8 | VIVA PREPARATION — Every question they will ask you

What is a Viva?

- A viva is a face-to-face oral exam where a professor asks you questions about your project.
- You must explain: architecture, algorithms, code, AI techniques, and future improvements.
- Practice answering out loud. Speaking and typing/writing are very different skills.

Q: What is your project about?

A: I built a Smart Email Management AI Agent that connects to Gmail using Python, reads emails, classifies them as Urgent/Work/Personal/Finance/Spam using BERT, summarizes them using Gemini API, scores priority using rule-based reasoning, and drafts professional replies. The system is a goal-based utility agent built with LangChain.

Q: What type of AI agent is it?

A: It is a Goal-Based + Utility-Based hybrid agent. Goal-based because it has a clear goal — process every email and take the right action. Utility-based because it assigns a priority score (utility value) from 1 to 10 to every email to decide importance.

Q: Which search algorithm did you use?

A: I implemented BFS — Breadth-First Search — to traverse email conversation threads. Each email is a node, thread replies are edges. BFS explores all replies at the same depth before going deeper, helping the agent understand full conversation context.

Q: What is OAuth2? Why did you use it?

A: OAuth2 is a secure authorization protocol. Instead of giving our program the Gmail password, we get a limited-access token from Google. This token only allows reading emails, not deleting or sending. It expires and can be refreshed automatically. This is more secure than storing passwords.

Q: What is BERT? How did you use it?

A: BERT stands for Bidirectional Encoder Representations from Transformers. It is a pre-trained deep learning model from Google that understands text context from both left and right directions. I used the distilbert-base-uncased model from Hugging Face to classify emails into 5 categories without training from scratch.

Q: What is LangChain?

A: LangChain is a framework for building AI agents. It allows defining tools that the agent can call. My agent has 4 tools: Classify, Summarize, Reply, and Priority. The agent receives an email and autonomously decides which tools to call and in what order.

Q: How did you test your agent?

A: I ran 3 formal test scenarios: an urgent faculty email, a spam promotional email, and a placement announcement email. For each, I recorded the input, expected output, and actual output. The agent correctly handled all 3 scenarios.

Q: What are the limitations?

A: Three main limitations: First, Gmail API has rate limits — 100 emails per minute. Second, BERT classification is ~85% accurate, some emails may be misclassified. Third, the system requires internet connection for Gemini API calls.

Q: Did you use AI tools? How?

A: Yes, I used Claude AI for architecture guidance and understanding concepts, and Gemini API as a core component for summarization and reply generation. All code was written by me and I can explain every line.

Q: What would you add in the future?

A: Three improvements: multi-language email support, Google Calendar integration to auto-schedule meetings from emails, and a mobile app using Flutter for on-the-go email management.

CHAPTER 9 | SUBMISSION CHECKLIST — Do not submit without ch

9.1 Complete Submission Checklist

ZIP FILE CONTENTS

- ☐ project/src/gmail_auth.py
- ☐ project/src/email_fetcher.py
- ☐ project/src/email_cleaner.py
- ☐ project/src/classifier.py
- ☐ project/src/summarizer.py
- ☐ project/src/reply_generator.py
- ☐ project/src/priority_engine.py
- ☐ project/src/bfs_thread.py
- ☐ project/src/agent.py
- ☐ project/src/app.py (Streamlit UI — bonus)
- ☐ project/data/emails.json (sample only, NOT your real emails)
- ☐ project/README.md
- ☐ project/requirements.txt

PDF REPORT (8-10 pages)

- ☐ Problem Statement (1 page)
- ☐ Agent Architecture with diagram (1 page)
- ☐ AI Techniques section — BFS, BERT, Rules, LLM (2-3 pages)
- ☐ Implementation Details with code snippets (1-2 pages)
- ☐ Results and 3 Test Scenarios (1 page)
- ☐ Limitations, Conclusion, References (1 page)
- ☐ AI Usage Declaration (last page — mandatory)

DEMO VIDEO (3-5 minutes)

- ☐ Screen recording — use OBS Studio (free)
- ☐ Show: problem explanation (30 seconds)
- ☐ Show: running the code on real emails (2 min)
- ☐ Show: agent decision making — which tool it called and why (1 min)
- ☐ Show: Streamlit UI if built (30 seconds)
- ☐ Speak clearly and explain what is happening on screen

THINGS TO NOT FORGET

- ☐ Remove credentials.json and token.json from ZIP (do NOT share these)
- ☐ Add a sample emails.json with fake/dummy emails — not your real ones
- ☐ README.md must have: description, installation steps, how to run, example input
- ☐ requirements.txt must list ALL libraries used
- ☐ AI Declaration must list every AI tool used and what it was used for

Final Mark Maximization Tips

- Build Streamlit UI — guaranteed +5 bonus marks. Takes only 2-3 hours in Week 6.
- Add BFS explicitly in report — it satisfies the mandatory search algorithm requirement.
- Show originality — mention which parts you built from scratch vs used libraries for.
- In viva, explain every line confidently — professors grade on understanding, not just output.
- Submit 2 days before deadline — avoid last-minute technical issues.
- Demo video must show agent making a DECISION — not just output. Show the reasoning.

DO NOT DO THESE

- Do not submit your real credentials.json or token.json — never share API keys.
- Do not copy-paste entire code from GitHub without understanding it.
- Do not skip the AI Usage Declaration — it is mandatory and marks are deducted.
- Do not submit a video that just shows code running — explain what is happening.
- Do not have spelling errors in your report — proofread before submission.

You have everything you need, Aswarth. The plan is clear. The code is ready. The report structure is given. Follow this guide week by week and you will build something that is genuinely impressive — not just for marks, but as a real project you can show in placements and interviews.

Good luck. Build it with understanding, not just code.